STLmc User Manual

Geunyeol Yu
Pohang University of Science and Technology, Pohang, Korea

August 2026

Contents

1 Introduction

STLmc is a bounded model checker based on SMT for signal temporal logic (STL) properties of hybrid systems [7]. It verifies whether all behaviors of a hybrid system satisfy an STL property up to a discrete bound, a time bound, and a robustness threshold specified by the user. The tool supports hybrid systems with discrete transitions and continuous dynamics, including nonlinear ordinary differential equations (ODEs).

STLmc provides a modeling language for hybrid automata and STL properties, a configurable command line interface for model checking and reachability analysis, and support for inspecting counterexamples and reachability witnesses. It can use Z3 [2], Yices2 [3], or dReal [4] as its underlying SMT solver.

Figure 1 gives an overview of the STLmc architecture. The tool takes a hybrid automaton H, an STL property φ, a discrete bound N, a time bound τ, and a robustness threshold ϵ. The parser reads the model, property, and analysis configuration. The model checking engine reduces robust STL model checking to Boolean STL model checking through ϵ-strengthening and constructs bounded SMT queries. The queries can be solved directly or using an optionally parallelized two step procedure, which is particularly useful for ODE models.

Model checking results are reported as satisfied, violated, or unknown. A satisfied result guarantees the property up to the given bounds and robustness threshold, while an unknown result is inconclusive. Reachability results are reported as reachable, unreachable within the given bounds, or unknown. The tool can export a violating execution or reachability witness as text or visualize it as signal and robustness plots.

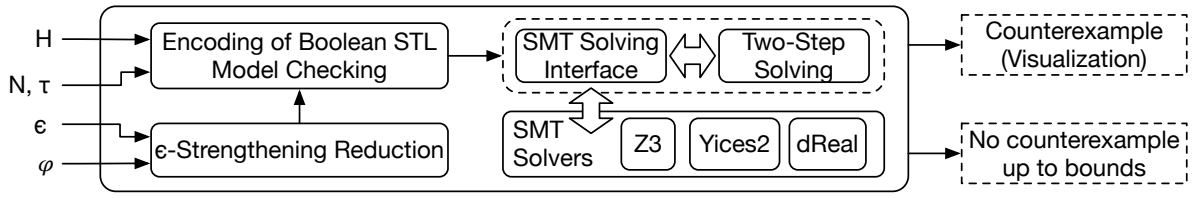

Figure 1: Architecture of STLmc.

The tool is available at `https://stlmc.github.io/`. Further details about the design, model checking algorithm, and evaluation are presented in the CAV 2022 tool paper [7].

2 Installation

STLmc supports CPython 3.8 through 3.14 and requires a current version of `pip`. Install STLmc from the Python Package Index as follows:

```
$ python3 -m pip install --upgrade pip
$ python3 -m pip install stlmc
```

The installation provides three commands. `stlmc` runs the model checker, and `stlmc-vis` analyzes its results. The third command, `stlmc-install-solvers`, manages solver setup. Check the solver prerequisites as follows:

```
$ stlmc-install-solvers --check
```

Running `stlmc-install-solvers` installs missing prerequisites. A solver name—`z3`, `yices`, or `dreal`—may be given to install only that solver. The installer obtains the native Yices library on Ubuntu, Debian, and macOS, and installs dReal3 in a user data directory on macOS or 64-bit Linux. A separately installed dReal3 executable can instead be specified as described in Section 6.

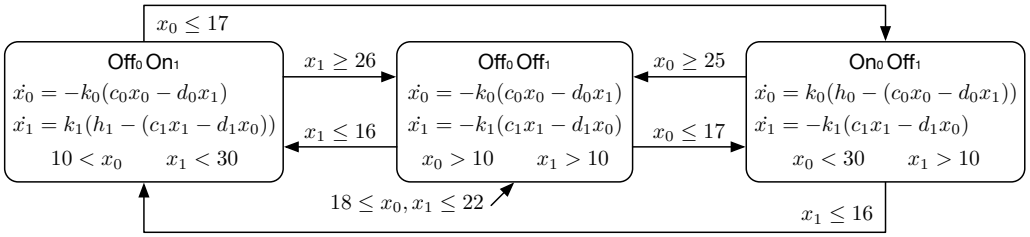

Figure 2: Hybrid automaton for the networked thermostat model.

Gnuplot is optional and is required only for generating result plots in PDF format. On Ubuntu or Debian, install it with:

```
$ sudo apt install gnuplot
```

On macOS, use Homebrew:

```
$ brew install gnuplot
```

The latest installation instructions are available at `https://stlmc.github.io/download/`.

3 Running example

This section introduces a networked thermostat model adapted from Bae and Gao [1], building on the classical thermostat hybrid automaton [5]. Two rooms are connected by an open door, and each has a heater that can be either on or off. For room $i \in \{0, 1\}$, let x_i denote its temperature and $q_i \in \{\mathsf{On}, \mathsf{Off}\}$ its heater mode. Heat transfer through the doorway couples the two rooms. The continuous dynamics of room i are given by:

$$\dot{x}_i = \begin{cases} k_i(h_i - (c_i x_i - d_i x_{1-i})) & (\mathsf{On}) \\ -k_i(c_i x_i - d_i x_{1-i}) & (\mathsf{Off}) \end{cases}$$

The positive constants k_i, h_i, c_i, and d_i depend on the size of the room, the power of the heater, and the size of the open door.

Initially, both heaters are off and $18 \le x_0, x_1 \le 22$. The controller turns on a heater when its room becomes too cold and turns it off when the room becomes warm. The three controller modes shown in Figure 2 ensure that at most one heater is on.

We use the following STL properties throughout the manual:

- $\mathsf{f1} = \Diamond_{[0,15]}((x_0 \ge 14)\, \mathbf{U}_{[0,\infty)}(x_1 \le 19))$: at some point within the first 15 time units, the temperature of the first room is at least 14 and stays at or above that value until the temperature of the second room reaches 19 or below.
- $\mathsf{f2} = \Box_{[2,4]}((x_0 - x_1 \ge 4) \rightarrow \Diamond_{[3,10]}(x_0 - x_1 \le -3))$: between time 2 and time 4, whenever the temperature of the first room exceeds that of the second by at least 4, the temperature of the second exceeds that of the first by at least 3 after a delay of 3 to 10 time units.
- $\mathsf{f3} = \Box_{[0,10]}((x_0 > 23)\, \mathbf{R}_{[0,\infty)}(x_0 - x_1 \ge 4))$: at every point during the first 10 time units, the temperature of the first room exceeds that of the second by at least 4 through the point at which it exceeds 23. If it never exceeds 23, the difference remains at least 4 indefinitely.

The complete STLMC input for this example is presented in Section 4. Its analysis and configuration are described in Sections 5 and 6.

```
int on0;              int on1;
[10, 35] x0;          [10, 35] x1;
const k0 = 0.015;     const k1 = 0.045;
const h0 = 100;       const h1 = 200;
const c0 = 0.98;      const c1 = 0.97;
const d0 = 0.01;      const d1 = 0.03;

{ mode: on0 = 0;      on1 = 0;
  inv:  x0 > 10;      x1 > 10;
  flow:
    d/dt[x0] = -k0 * (c0*x0 - d0*x1);
    d/dt[x1] = -k1 * (c1*x1 - d1*x0);
  jump:
    x0 <= 17 => (and (on0' = 1)
      (on1' = on1) (x0' = x0) (x1' = x1));
    x1 <= 16 => (and (on1' = 1)
      (on0' = on0) (x0' = x0) (x1' = x1));
}
{ mode: on0 = 0;      on1 = 1;
  inv:  10 < x0;      x1 < 30;
  flow:
    d/dt[x0] = -k0 * (c0*x0 - d0*x1);
    d/dt[x1] = k1 *
      (h1 - (c1*x1 - d1*x0));
  jump:
    x0 <= 17 => (and (on0' = 1)
      (on1' = 0) (x0' = x0) (x1' = x1));
    x1 >= 26 => (and (on1' = 0)
      (on0' = on0) (x0' = x0) (x1' = x1));
}
{ mode: on0 = 1;      on1 = 0;
  inv:  x0 < 30;      x1 > 10;
  flow:
    d/dt[x0] = k0 *
      (h0 - (c0*x0 - d0*x1));
    d/dt[x1] = -k1 * (c1*x1 - d1*x0);
  jump:
    x1 <= 16 => (and (on0' = 0)
      (on1' = 1) (x0' = x0) (x1' = x1));
    x0 >= 25 => (and (on0' = 0)
      (on1' = on1) (x0' = x0) (x1' = x1));
}
init: on0 = 0; 18 <= x0; x0 <= 22;
      on1 = 0; 18 <= x1; x1 <= 22;

proposition:
  [p1]: x0 - x1 >= 4;
  [p2]: x0 - x1 <= -3;


goal:
  [f1]: <>[0,15] (x0 >= 14 U[0,inf) x1 <= 19);
  [f2]: [][2,4] (p1 -> <>[3,10] p2);
  [f3]: [][0,10] (x0 > 23 R[0,inf) p1);
```

Figure 3: The model file `thm.model`.

4 Modeling language

STLmc provides a modeling language for hybrid systems. Its model file format, inspired by dReach [6], consists of the following sections, which must appear in this order:

- *Variable declarations* define state and mode variables and constants.
- *Mode definitions* specify invariants, continuous flows, and jumps.
- *Initial conditions* define the initial states.
- *State propositions* optionally name conditions used in STL formulas.
- *STL properties* define temporal requirements over model states.

Figure 3 presents the complete STLmc model file for the running example. The file declares variables that record whether each heater is on or off, the two room temperatures, and the constants for their continuous dynamics. It also defines the invariants and switching behavior of three modes, the initial states, and the state propositions and STL properties.

4.1 Variable declarations

STLmc represents a hybrid state using mode and continuous variables. *Mode variables* remain constant during continuous evolution and may be updated by jumps, whereas *continuous variables* have real values and evolve according to the flow of the current mode. Together, their values describe a state of the hybrid automaton. A model may also declare named constants whose values do not change.

Mode variables may be declared with one of three types: `bool` variables with `true` and `false` values, `int` variables with integer values, and `real` variables with real values. For example, the following declares one mode variable of each type:

```
bool b;    int i;    real r;
```

Variable names and goal or proposition labels must begin with a letter and may contain only letters and digits. Language keywords cannot be used as identifiers; these include `t` and `inf`, which denote time and infinity, respectively.

Continuous variables are declared with domain intervals. Domains can be bounded or unbounded intervals of real numbers, including open, closed, and half open intervals. An infinite endpoint is written as `-inf` or `inf` with an open parenthesis. A singleton domain is written with one value in square brackets. For example, the following declares four continuous variables with different domains:

```
[0, 50] x;    (-1.1, 1) y;    (-inf, inf) z;    [3] w;
```

Constants are introduced with the `const` keyword. Constants can have the Boolean literals `true` and `false` or a numeric literal. Numeric literals may use integer, decimal, or scientific notation; an arithmetic expression such as `1/3` is not accepted in a constant declaration. For example, the following declares a numeric constant `k1` and a Boolean constant `heaterEnabled`:

```
const k1 = 0.015;    const heaterEnabled = true;
```

Mode variable, continuous variable, and constant declarations may be interleaved, but they must all precede the first mode block.

4.2 Mode definitions

In STLMC, a *mode block* describes one or more discrete configurations together with their invariant, continuous dynamics, and outgoing transitions. A model may contain multiple mode blocks, each with four components:

```
{
    mode: ...
    inv:  ...
    flow: ...
    jump: ...
}
```

The components must appear in the order shown. The `mode` and `flow` components contain at least one condition or equation. The `inv` and `jump` components may be empty.

A mode block applies to the current state valuations selected by its `mode` component. Its `inv` component gives conditions that must hold throughout continuous evolution, while its `flow` component defines that evolution. The `jump` component contains outgoing guard and reset rules.

A `mode` component contains Boolean state conditions separated by semicolons, normally over mode variables. The conjunction of these conditions represents a set of modes. Suppose that `b` is a Boolean mode variable and `i` is an integer mode variable. The following conditions select the two valuations $\{(true, 1), (true, 2)\}$:

```
b = true;    i > 0;    i < 3;
```

An `inv` component contains Boolean state conditions separated by semicolons, typically over continuous variables. The conjunction of these conditions is the invariant of the mode block and must hold throughout its continuous evolution. For example, the following declares an invariant condition for two continuous variables `x` and `y`:

```
x < 30;    y > -0.5;
```

A `flow` component contains either a system of ordinary differential equations (ODEs) or closed form dynamics. The two forms cannot be mixed within one mode block. In STLMC, a system of ODEs over continuous variables $x_1, \ldots, x_n$ is written as a sequence of equations of the following form, separated by semicolons. Each e_i is an expression over declared variables and constants:

```
d/dt[x_1] = e_1(x_1, ..., x_n) ;
...
d/dt[x_n] = e_n(x_1, ..., x_n) ;
```

For example, the following ODEs define the continuous dynamics $\dot{x} = \sin(y)$ and $\dot{y} = y^2$:

```
d/dt[x] = sin(y);  d/dt[y] = y ** 2;
```

A closed form flow is written as a set of continuous functions, parameterized by a time variable `t` and the initial values $x_1(0), \ldots, x_n(0)$ for the mode block:

```
x_1(t) = e_1(t, x_1(0),...,x_n(0)) ;
...
x_n(t) = e_n(t, x_1(0),...,x_n(0)) ;
```

For example, the same variables may instead be defined by the following closed form expressions, $x(t) = t^2 + 3t + x(0)$ and $y(t) = t + y(0)$:

```
x(t) = t ** 2 + 3 * t + x(0);  y(t) = t + y(0);
```

A `jump` component may contain multiple rules of the form *guard* `=>` *reset*, each describing a possible transition. A transition is enabled when the current state satisfies its guard. The reset constrains primed variables, which represent the state after the jump, using expressions over current state variables and constants.

A reset should explicitly constrain every mode and continuous variable. To preserve a value, write `x' = x`; an omitted variable is unconstrained after the jump. For example, the following defines a jump with four variables `b`, `i`, `x`, and `y` (declared above):

```
(and (i = 1) (x > 10)) =>
  (and (b' = false) (i' = 3) (x' = x) (y' = 0));
```

4.3 Initial conditions

Every model must contain exactly one `init` section after its mode blocks. The section contains one or more Boolean state conditions separated by semicolons over declared mode and continuous variables. Arithmetic expressions in these conditions may also use numeric constants declared with `const`. The conditions are interpreted conjunctively and define the set of initial states.

Initial conditions use unprimed variables. A variable omitted from the `init` section is not assigned a default value; it remains unconstrained except for its declared domain and the conditions of the selected mode block. For example, the following constrains `b`, `i`, `x`, and `y`:

```
init:
not b;    i = 0;    19.9 <= x;    y = 0;
```

4.4 State propositions and STL properties

Named state propositions may be declared in the optional `proposition` section before the `goal` section. They provide short names for conditions that occur repeatedly in STL formulas. Each proposition defines a Boolean state condition; it cannot contain temporal operators or refer to another named proposition. For example:

```
proposition:
[p1]: x0 - x1 >= 4;
[p2]: x0 - x1 <= -3;
```

Every proposition referenced by an STL formula must be declared in the `proposition` section.

The required `goal` section declares the STL properties or reachability queries to be analyzed. An STL property may be declared with or without a label. A label is written in square brackets before the colon and uniquely identifies the property within the model file.[1] For example, the following declares two labeled STL properties from the running example using atomic conditions directly in the formulas:

```
goal:
[f1]: <>[0, 15](x0 >= 14 U[0, inf) x1 <= 19);
[f2]: [][2, 4]((x0 - x1 >= 4) -> <>[3, 10] (x0 - x1 <= -3));
```

The second formula can be written using the propositions declared above:

```
goal:
[f2]: [][2, 4](p1 -> <>[3, 10] p2);
```

The temporal operators are `[]` (always), `<>` (eventually), `U` (until), and `R` (release). Boolean conjunction and disjunction are written as `and` and `or`, negation as `~`, and implication as `->`; `true` and `false` are Boolean constants. Parentheses are recommended when Boolean and temporal operators are combined.

Temporal intervals may be open, closed, or half open. An unbounded interval uses `inf` as an open upper endpoint, as in `[0, inf)`. A singleton interval is written as `=c`, as in `[]=3 p1`.

In addition to STL properties, the `goal` section accepts an unlabeled goal written with the `reach` keyword followed by a state condition.[2]

```
goal:
reach (x >= 10);
```

4.5 Comments

A line comment begins with `#` and continues to the end of the line. A block comment begins and ends with three single quotation marks. Both forms may appear wherever whitespace is allowed in a model file.

```
# This is a line comment.
''' This is a
    block comment. '''
```

5 Analysis

This section provides an overview of the analyses supported by STLmc and shows how to run the tool and interpret its output. Two types of analysis are available:

- *Robust STL model checking* searches the bounded executions of a model for a violation of an STL property at the given robustness threshold.
- *Reachability analysis* asks whether a bounded execution can reach a state satisfying a given condition. A query can be declared in the model with `reach`; alternatively, `-reach` treats a selected state goal without temporal operators as a reachability target.

Optional analysis features, such as the two step procedure and parallel solving, can be enabled through command line arguments or configuration files. The SMT solver may also be specified explicitly; otherwise, STLmc selects a compatible solver automatically. Section 6 describes these settings, and Section 8 summarizes the solver capabilities.

[1] Labels can be used to select individual goals for analysis; see Section 6.

[2] The corresponding reachability analysis and the `reach` argument are described in Section 5.

5.1 Running an analysis

The command line must specify a model file. An analysis also requires a discrete bound and a time bound, which may be supplied as command line arguments or inherited from configuration files. Other analysis settings may be provided in either form as well. Figure 4 shows the command line syntax, and Section 6 describes the available arguments, configuration syntax, and precedence rules.

```
$ stlmc [path to model file] \
        -bound [int] -time-bound [number] \
        -threshold [number] -goal [name] ...
```

Figure 4: Command line syntax for running an analysis.

Before solving, STLmc prints the model, goal, solver, analysis method, execution mode, bounds, threshold, and query type. In model checking, the discrete bound limits the number of mode changes and STL variable points. In reachability analysis, it limits the number of jumps, so bound zero examines the initial continuous segment. The applicable bound kind is included in the printed settings.

Model checking finishes with `satisfied`, `violated`, or `unknown`. Reachability analysis instead reports `reachable`, `unreachable`, or `unknown`. A violating execution is a counterexample, while an execution reaching the target is a witness. An `unknown` result is inconclusive.

One step and two step analysis can each use symbolic or explicit path exploration. Symbolic exploration leaves mode and jump choices in the SMT encoding, whereas explicit exploration enumerates transition sequences. Batching is available for explicit one step and two step analysis, whereas parallel execution applies only to two step refinement checks. These settings are described in Section 6.

When a counterexample or witness is found, `-visualize` writes result data and a visualization configuration. Section 7 describes these artifacts and their output formats. Generated solver queries can also be retained with `-save-smt2`; its output directory is selected with `-smt2-dir`.

For example, Figure 5 analyzes `f2` from the running example with maximum discrete bound 5, time bound 25, and threshold $\epsilon = 2$. The command uses dReal and the parallel two step method and finds a counterexample at bound 2.

```
$ stlmc ./thm.model -bound 5 -time-bound 25 \
        -threshold 2 -goal f2 -solver dreal \
        -two-step -parallel -visualize
STLMC v1.0.1.dev3
Signal Temporal Logic Model Checker
...
Result
  status      : violated at bound 2
  time        : 7.463s
  formula     : ([][2.0,4.0] (((x0 - x1) >= 4) ->
                  (<>[3.0,10.0] ((x0 - x1) <= -3))))
  artifacts   : thm_b2_f2_dreal.counterexample
                thm_b2_f2_dreal.cfg
```

Figure 5: Abridged output for the analysis of `f2` in the running example.

6 Configuration

Analysis settings can be supplied as command line arguments or stored in configuration files. Command line arguments are convenient for individual runs, while configuration files provide reusable settings for a model or an individual goal. This section describes both forms and how their values are combined.

6.1 Command line arguments

The command line must specify a model file. The configuration must also provide `bound` and `time-bound`, either through command line arguments or through one of the configuration layers. All other configuration files and command line overrides are optional.

```
$ stlmc [path to model file] \
        -default-cfg [path to default config file] \
        -model-cfg [path to model config file] \
        -model-specific-cfg [path to goal config file] \
        -bound [int] ...
```

Most analysis settings have both a configuration parameter and a command line form. A command line argument is formed by adding a leading hyphen to the parameter name; for example, `bound` becomes `-bound`. Boolean arguments such as `-two-step`, `-parallel`, and `-save-smt2` enable the corresponding option. To disable a Boolean value inherited from a configuration file, set it to `"false"` in a configuration file of higher priority.

By default, STLmc analyzes every STL goal declared in the model. The `-goal` argument selects an individual goal by its label.[3]

Individual command line values override every configuration file. Invalid solver and expression combinations are described in Section 8. The purpose and behavior of the analysis arguments are described in Section 5.

6.2 Configuration files

A configuration file has the extension `.cfg`. Parameters common to all solvers are declared in a `common` section, while parameters for individual solvers are declared in `z3`, `yices`, and `dreal` sections. Figure 6 shows the default configuration included with STLmc, and Table 1 summarizes the `common` parameters.

6.2.1 Common parameters

The parameters in Table 1 are written within the braces of the `common` section. There are two mandatory parameters and sixteen optional parameters. The mandatory parameters are `bound` and `time-bound`; an analysis cannot start until both have been supplied by one of the configuration layers or the command line.

The parameter names are the command line argument names without their leading hyphen. Numeric values are written directly, while Boolean values are quoted as `"true"` or `"false"`. The `smt2-dir` value is used only when `save-smt2` is enabled. See Section 5 for the overview of the supported analyses and the command line workflow.

The `path-strategy` parameter is independent of `two-step`, so all four combinations of one step or two step analysis and symbolic or explicit exploration are available. `solver-batch-size` limits candidates per solver query. In two step analysis, `core-minimize-attempts` controls unsat core minimization; its default value uses only the initial core. `concrete` retains a complete Boolean and discrete assignment instead of applying unsat core generalization.

[3] Goal selection is unavailable if the model contains an unlabeled STL property or a reachability query, because reachability queries cannot have labels.

```
# basic section
common {
    # mandatory arguments
    # bound =
    # time-bound =

    threshold = 0.01
    solver = "auto"                 # dreal , yices , z3
    time-horizon = "time-bound"
    parallel = "false"
    visualize = "false"
    goal = "all"
    path-strategy = "symbolic"
    two-step = "false"
    concrete = "false"
    verbose = "false"               # print verbose messages
    reach = "false"
    parallel-core = 25
    solver-batch-size = 1
    core-minimize-attempts = 1
    save-smt2 = "false"
    smt2-dir = "smt2-logs"
}

# underlying solver
z3      {
    logic = "QF_NRA" # "QF_LRA"
}

yices   {
    logic = "QF_NRA" # "QF_LRA"
}

dreal {
    precision = 0.001
    ode-order = 20
    # ode-step is omitted to use dReal's automatic step control.
    executable-path = "dReal" # dReal3 executable name or path
}
```

Figure 6: The default configuration included with STLmc.

6.2.2 Parameters for individual solvers

The SMT solver is selected with the `solver` parameter. Solver capabilities and automatic selection are described in Section 8.

Parameters for an individual solver are written in a section named after it. The `z3` and `yices` sections each accept a `logic` parameter, either `QF_LRA` or `QF_NRA`. The `dreal` section accepts `precision`, `ode-order`, `ode-step`, and `executable-path`. The `ode-order` and `executable-path` parameters are mandatory in that section. `precision` sets the precision δ for δ-complete solving. `ode-order` sets the maximum Taylor expansion order, and `executable-path` gives either the dReal3 executable name or its path. With the default value `dReal`, STLmc searches the system path and then its user solver directory. The optional `ode-step` sets the numerical integration step; when it is omitted, dReal uses automatic step control.

Table 1: Parameters in the `common` section.

Name	Explanation	Default
`bound`	a discrete bound $N \in \mathbb{N}$	-
`time-bound`	a time bound $\tau \in \mathbb{Q}^+$	-
`threshold`	a robustness threshold $\epsilon \in \mathbb{Q}^+$	0.01
`solver`	solver (`auto`/`z3`/`yices`/`dreal`)	`auto`
`time-horizon`	a continuous segment duration bound	τ
`goal`	a list of STL goals to be analyzed	all
`path-strategy`	symbolic or explicit path exploration	`symbolic`
`two-step`	use the two step optimization	disabled
`parallel`	parallelize two step analysis	disabled
`parallel-core`	maximum workers for parallel two step solving	25
`visualize`	generate counterexample or witness data	disabled
`concrete`	retain complete scenarios in two step analysis	disabled
`reach`	check reachability instead of the negated STL goal	disabled
`solver-batch-size`	candidates combined in one solver query	1
`core-minimize-attempts`	unsat core minimization attempts	1
`save-smt2`	save generated SMT-LIB2 queries	disabled
`smt2-dir`	output directory used by `save-smt2`	`smt2-logs`
`verbose`	print more information about the execution	disabled

6.2.3 Configuration precedence

Parameters are combined in the following order, with each later source overriding values from earlier sources:

1. the default configuration included with STLmc;
2. a model configuration;
3. a model specific configuration; and
4. command line options.

The `default.cfg` file installed with the STLmc package provides every default except the mandatory `bound` and `time-bound`. A file supplied with `-default-cfg` replaces this default file. It must therefore provide all required sections and all mandatory solver parameters, in addition to values that may be supplied by later configuration layers.

Without `-model-cfg`, STLmc looks beside the model file for the configuration name formed from the part of the model file name before its first period; for example, `thm.model` causes `thm.cfg` to be loaded when that file exists. A model specific file is loaded only when it is supplied with `-model-specific-cfg`; it is not discovered automatically.

The model configuration stores reusable settings for a particular model. In the running example, `thm.cfg` sets bound 5, time bound 25, dReal, and the parallel two step algorithm for the model in Figure 2. Figure 7 shows this partial configuration; all omitted parameters retain their values from the default configuration.

A model specific configuration can override selected analysis parameters for an individual goal. For example, the configuration in Figure 8 selects the goal `f2`, sets $\epsilon = 2$, and changes the continuous segment duration bound. All other values are inherited from the model configuration and the default configuration.

```
# STLmc configuration
common { bound = 5
         time-bound = 25
         solver = "dreal"
         verbose = "true"
         two-step = "true"
         parallel = "true" }
```

Figure 7: A model configuration example.

```
# STLmc configuration
common { goal = f2
         threshold = 2
         time-horizon = 30 }
```

Figure 8: A model specific configuration example.

7 Counterexample and witness analysis

With `-visualize` enabled, STLmc records a violating model checking execution in a result file with the `.counterexample` extension. A successful reachability execution uses the `.witness` extension. The `stlmc-vis` command can inspect either artifact in two ways: it can export a textual account of the execution or visualize the state and robustness trajectories.

7.1 Text output

Text output provides a readable account of the counterexample without a visualization configuration file:

```
$ stlmc-vis [path to result file] -output text
```

The command writes the goal, bounds, initial condition, constants, mode sequence, state values, and jump candidates to a file formed by appending `.txt` to the result file name. For example, the following command writes `thm_b2_f2_dreal.counterexample.txt`:

```
$ stlmc-vis thm_b2_f2_dreal.counterexample -output text
```

Figure 9 shows an excerpt from a generated file. Its header identifies the goal, expanded formula, bound, and solver precision. Subsequent sections list the declared constants, domains of continuous variables, and initial condition. Each `Bound` block records one trajectory segment: its time interval, active mode, invariant, continuous dynamics, and state values at the two endpoints. Except for the last segment, the block also lists the next mode and the jump candidates from the current segment to the next.

For example, the first segment in Figure 9 runs from time 0 to approximately 14.62 with both heaters off. Over this segment, x_0 changes from approximately 20.42 to 16.50, while x_1 changes from approximately 18.42 to 10.00. The arrow therefore separates the value at the start of the segment from the value at its end. The next mode turns on the second heater. The line containing `=>` pairs the guard $x_1 \leq 16$ with the reset that performs this transition while preserving x_0 and x_1. Ellipses in the figure mark content omitted from the displayed excerpt; they are not part of the generated file. The file repeats the `Bound` block for every trajectory segment and omits `next mode` and `jump candidates` from the last block.

```
Counterexample
==============
goal: f2
formula: ([][2.0,4.0] (((x0 - x1) >= 4) ->
  (<>[3.0,10.0] ((x0 - x1) <= -3))))
bound: 2
delta: 2.0
Constants
---------
c0 = 0.98
c1 = 0.97
...
Variables
---------
x0: [10.0, 35.0]
x1: [10.0, 35.0]
Initial condition
-----------------
(and (on0 = 0) (18 <= x0) (x0 <= 22)
  (on1 = 0) (18 <= x1) (x1 <= 22))
Bound 0
-------
time: 0.000000 -> 14.615539
mode: (and (on0 = 0) (on1 = 0))
invariant:
  (and (x0 > 10) (x1 > 10))
flow:
  d/dt[x0] = ((- 0.015) * ((0.98 * x0) - (0.01 * x1)))
  d/dt[x1] = ((- 0.045) * ((0.97 * x1) - (0.03 * x0)))
state:
  x0: 20.42416 -> 16.502264
  x1: 18.42416 -> 10.000048
next mode: (and (on0 = 0) (on1 = 1))
jump candidates:
  (x0 <= 17) => (and (on0' = 1) (on1' = on1)
    (x0' = x0) (x1' = x1))
  (x1 <= 16) => (and (on1' = 1) (on0' = on0)
    (x0' = x0) (x1' = x1))
...
```

Figure 9: An excerpt from a text counterexample file.

7.2 Visualization

HTML and PDF output plot the state and robustness trajectories in a counterexample. Both formats use a visualization configuration file:

```
$ stlmc-vis [path to result file] \
            -cfg [path to configuration file] \
            -output [html|pdf]
```

If `-cfg` is omitted, `stlmc-vis` automatically looks in the current directory for a configuration file whose name matches the result file. For example, `thm_b2_f2_dreal.counterexample` uses `thm_b2_f2_dreal.cfg`. The `-output` argument overrides the format recorded in that file. HTML is the default format, while PDF output requires Gnuplot.

The visualization configuration records:

- the continuous variables and STL subformula labels available for plotting;
- the output format; and
- groups of variables to be plotted together.

The states of variables in a group are plotted on one graph. Only variables of the same type can be grouped together; for example, a group containing `x0` and `f2` is rejected because their types are real and Boolean, respectively. Variables not assigned to a group are grouped together according to their type. Figure 10 shows the thermostat visualization configuration.

Consider the counterexample for `f2` generated by the analysis in Figure 5 and the visualization configuration below. The following command generates the PDF plots shown in Figure 11.

```
$ stlmc-vis thm_b2_f2_dreal.counterexample -cfg thm_vis.cfg
```

```
{   # continuous variables: x0, x1
    # STL subformula labels:
    # f2_0 --> [][2, 4]((x0 - x1 >= 4) -> <>[3,10](x0 - x1 <= -3))
    # f2_1 --> (x0 - x1 >= 4) -> <>[3,10](x0 - x1 <= -3)
    # f2_2 --> not (x0 - x1 >= 4)
    # f2_3 --> <>[3,10](x0 - x1 <= -3)
    # f2_4 --> x0 - x1 <= -3
    # f2_5 --> x0 - x1 >= 4

    output = pdf # html
    group { (x0, x1), (f2_0, f2_1), (f2_2, f2_3), (f2_4, f2_5) }
}
```

Figure 10: A visualization configuration example.

At time 0, the robustness degree of `f2` is below ϵ because $\texttt{f2}_1$ falls below the threshold between time 2 and time 4. Over that period, both $\texttt{f2}_2$ and $\texttt{f2}_3$ are below the threshold. In particular, $\texttt{f2}_3$ evaluates the atomic condition $x_0 - x_1 \leq -3$ from 3 to 10 time units later, so the relevant times extend from 5 to 14. In Figure 11, underscores are omitted from the plot legends, and the two atomic conditions are abbreviated as `p1` and `p2` for readability.

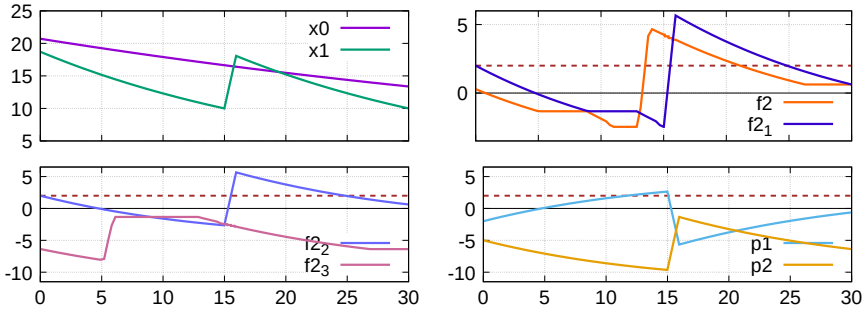

Figure 11: Visualization of a counterexample (horizontal dotted lines denote $\epsilon = 2$).

8 SMT solver support

STLmc supports Z3 [2], Yices2 [3], and dReal [4]. Compatibility with a solver depends on the expressions that appear throughout the model file, including expressions in flows, invariants, jump conditions, initial conditions, state propositions, and STL properties. Table 2 summarizes the expression constructs that STLmc can translate for each solver.

Table 2: Model expression constructs supported by each SMT solver.

Category	Operators and functions	Z3	Yices2	dReal3
Boolean operators	`and`, `or`, `not`, `~`	✓	✓	✓
Comparisons	`>`, `>=`, `<`, `<=`, `=`, `!=`	✓	✓	✓
Arithmetic	`+`, `-`, `*`, `/`, and unary `-`	✓	✓	✓
Constant powers	`x ** n`, for a constant integer `n >= 0`	✓	✓	✓
Other powers	`x ** y`, for a fractional, negative, or symbolic `y`	–	–	✓
Functions	`sqrt`, `sin`, `cos`, `tan`, `arcsin`, `arccos`, `arctan`	–	–	✓

In Table 2, a check mark indicates that STLmc can translate the construct for the solver; a dash indicates that STLmc does not support the combination. A check mark does not guarantee that every resulting nonlinear problem can be solved within a particular time. Division by an expression that may be zero is also subject to the arithmetic semantics of the selected solver. Z3 and Yices2 accept only constant integer powers greater than or equal to zero; fractional, negative, and symbolic powers require dReal.

8.1 Solver selection and compatibility

With `solver = "auto"`, STLmc examines the expressions in the model and all declared STL goals. It selects dReal if the model contains ODEs or if any expression uses a transcendental function or a fractional, negative, or symbolic exponent. Otherwise, STLmc uses Yices2 and selects `QF_LRA` or `QF_NRA` according to whether the model dynamics are linear or nonlinear.

When a solver is selected explicitly, every expression in the model and the selected goal must be supported by that solver. If they are incompatible, the analysis cannot proceed; users should manually select a compatible solver from Table 2 or use `solver = "auto"`.

8.2 dReal function names

The model language uses `arcsin`, `arccos`, and `arctan`. STLmc translates them to `asin`, `acos`, and `atan`, respectively, for the SMT-LIB2 syntax accepted by dReal3. For compatibility with that solver, `tan(x)` is translated to `sin(x) / cos(x)`.

References

[1] Kyungmin Bae and Sicun Gao. Modular smt-based analysis of nonlinear hybrid systems. In *Proceedings of the 17th Conference on Formal Methods in Computer-Aided Design*, FMCAD '17, pages 180–187, Austin, TX, 2017. FMCAD.

[2] Leonardo De Moura and Nikolaj Bjørner. Z3: an efficient SMT solver. In *Proc. TACAS*, volume 4963 of *LNCS*, pages 337–340. Springer, 2008.

[3] Bruno Dutertre. Yices 2.2. In Armin Biere and Roderick Bloem, editors, *Proc. CAV*, volume 8559 of *LNCS*, pages 737–744. Springer, 2014.

[4] Sicun Gao, Soonho Kong, and Edmund M. Clarke. dReal: An SMT solver for nonlinear theories over the reals. In *Proc. CADE*, volume 7898 of *LNCS*, pages 208–214. Springer, 2013.

[5] Thomas A. Henzinger. The theory of hybrid automata. In *Verification of Digital and Hybrid Systems*, volume 170 of *NATO ASI Series*, pages 265–292. Springer, 2000.

[6] Soonho Kong, Sicun Gao, Wei Chen, and Edmund M. Clarke. dReach: δ-reachability analysis for hybrid systems. In *Proc. TACAS*, volume 7898 of *LNCS*, pages 200–205. Springer, 2015.

[7] Geunyeol Yu, Jia Lee, and Kyungmin Bae. STLmc: Robust STL model checking of hybrid systems using SMT. In *Computer Aided Verification*, volume 13371 of *Lecture Notes in Computer Science*, pages 524–537. Springer, 2022.